Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 9:
Tabella dei Simboli e Compilazione Modulare

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Analisi dei Simboli con `nm` 2
3.2 Compilazione Modulare 2
3.3 Istruzioni di Esecuzione 3

4 Risultati **3**
4.1 Ispezione dei Segmenti (`size`) 3
4.2 Effetto della Keyword `static` 3

5 Conclusioni e Sviluppi Futuri **3**

1 Problem Statement

L'Hands-On 9 analizza i meccanismi interni alla trasformazione del codice sorgente C in binari esequibili in formato **ELF** (*Executable and Linkable Format*). L'obiettivo è comprendere come il compilatore e il linker gestiscano la visibilità degli identificatori attraverso la **Tabella dei Simboli**. Il sistema deve permettere la distinzione tra simboli globali, statici ed esterni, analizzando l'impatto delle direttive di compilazione sull'organizzazione della memoria e sulla risoluzione dei riferimenti incrociati in progetti multi-file.

2 Stato dell'Arte

La pipeline di compilazione moderna e la gestione dei simboli sono i pilastri del software engineering modulare.

- **Lifecycle del Simbolo**: Dall'analisi lessicale alla generazione del codice oggetto, ogni variabile o funzione viene catalogata. Il linker utilizza queste informazioni per unire i moduli (`.o`) in un unico spazio di indirizzamento.
- **Dynamic Linking vs Static Linking**: Mentre il linking statico include tutte le librerie nel binario, quello dinamico rimanda la risoluzione a runtime tramite la *Procedure Linkage Table* (PLT) e la *Global Offset Table* (GOT), permettendo il risparmio di spazio e memoria.
- **Sezioni ELF**: Un binario è diviso in segmenti logici (`.text`, `.data`, `.bss`, `.rodata`). La distinzione tra essi determina se un simbolo risiede in memoria a sola lettura o se richiede inizializzazione esplicita.

3 Metodologia ed Implementazione

L'implementazione si focalizza sull'uso di tool di introspezione binaria nel contesto di un Makefile strutturato.

3.1 Analisi dei Simboli con `nm`

È stata creata un'architettura multi-file (`main.c`, `ex1`$_symbols.c)perosservareilcomportamentodeisimbol$

3.2 Compilazione Modulare

L'uso di un `Makefile` ha permesso di automatizzare la generazione dei file oggetto e la verifica dei segmenti:

Snippet Makefile

```
all: main
main: main.o symbols.o
    $(CC) -o $@ $^
    size $@ # Verifica segmenti memoria
```

3.3 Istruzioni di Esecuzione

L'Hands-On utilizza un Makefile per gestire la tabella dei simboli:

Makefile

```
$ make all
$ ./programma
```

4 Risultati

La validazione ha confermato la corretta risoluzione dei simboli esterni durante la fase di linking.

4.1 Ispezione dei Segmenti (`size`)

L'analisi tramite `size` ha dimostrato che le variabili globali non inizializzate vengono collocate nel segmento `BSS`, che occupa 0 byte sul disco ma viene espanso in RAM, ottimizzando la dimensione dell'eseguibile.

4.2 Effetto della Keyword `static`

È stato verificato che prefiggendo `static` a una variabile globale, il suo simbolo muta da globale (`D/B`) a locale (`d/b`), rendendola invisibile al linker e proteggendo l'incapsulamento del modulo.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 9 ha chiarito i passaggi fondamentali della generazione del software. La comprensione della tabella dei simboli è essenziale per il debugging di errori di tipo *Multiple Definition* o *Undefined Reference*.
Sviluppi futuri:

- **Reverse Engineering**: Utilizzo di `objdump` e `readelf` per ricostruire il flusso di esecuzione da binari privi di debug symbols (*stripped*).
- **Security Hardening**: Studio dell'impatto del *Link-Time Optimization* (LTO) e della *Position Independent Code* (PIC) sulla sicurezza e sulle prestazioni del binario finale.